NTNU
Kunnskap for en bedre verden

Department ELSYS

TFE4171- Design av digitale system 2

Verification of High-level Data Link Control (HDLC)

Authors:
Alexandru Carbuneanu
Ketevan Kutchava

Date

Table of Contents

List of Figures

List of Tables

1 Introduction

The current document contains the report on the verification on the High-level Data Link Control (HDLC) module. Is this document is structured into immediate assertions, concurrent assertions, discussions and clonclusion sections. In the sections immediate assertions it concurrent assertions, showed and descussed the details of each point of the specification hat has been decided to be developed as that type of assertion. In order to make sure that all the assertion were passed, one can run simulate.sh in the tb folder and look into the generated transcript. Alternatively, the transcript output can be found in the Appendix C. On the following table it is shown the specification list and the assertion type is was decided to be

Number	Name	Type
1	Correct data in RX buffer according to RX input. The buffer should contain up to 128 bytes (this includes the 2 FCS bytes, but not the flags).	Immediate
2	Attempting to read RX buffer after aborted frame, frame error or dropped frame should result in zeros.	Immediate
3	Correct bits set in RX status/control register after receiving frame. Remember to check all bits. I.e. after an abort the Rx Overflow bit should be 0, unless an overflow also occurred.	Immediate
4	Correct TX output according to written TX buffer.	Immediate
5	Start and end of frame pattern generation (Start and end flag: 0111 1110).	Immediate
6	Zero insertion and removal for transparent transmission.	Immediate
7	Idle pattern generation and checking (1111 1111 when not operating).	Immediate
8	Abort pattern generation and checking (1111 1110). Remember that the 0 must be sent first.	Immediate
9	When aborting frame during transmission, Tx AbortedTrans should be asserted.	Immediate
10	Abort pattern detected during valid frame should generate Rx AbortSignal.	Concurrent
11	CRC generation and checking.	Immediate
12	When a whole RX frame has been received, check if end of frame is generated.	Immediate
13	When receiving more than 128 bytes, Rx Overflow should be asserted.	Concurrent
14	Rx FrameSize should equal the number of bytes received in a frame (max. 126 bytes = 128 bytes in buffer – 2 FCS bytes).	Concurrent
15	Rx Ready should indicate byte(s) in RX buffer is ready to be read.	Concurrent
16	Non-byte aligned data or error in FCS checking should result in frame error.	Concurrent
17	Tx Done should be asserted when the entire TX buffer has been read for transmission.	Concurrent and Immediate
18	Tx Full should be asserted after writing 126 or more bytes to the TX buffer (overflow).	Immediate

Table 1: Test Cases for RX/TX Buffer and Frame Handling

2 Immediate assertions

Immediate assertions are executed based on the event during the simulation. They check specific property at the moment simulator reaches the point where this assertion is located. Immediate assertions are used for combinatorial check and during the simulation the tool treats them as if-else blocks (cited from Guide 2025).
This section covers assertions from the part A and part B of the project. Part A only asserted the behavior of the receive design, while part B incorporated both. In order to verify that the design matches the specification, we created an additional task to transmit a certain amount of bytes. Additionally, we added three tasks to verify abort, overflow, drop, non byte aligned and normal behavior that are used in the Receive task (code was provided in the part A).
In order to verify specifications from the part B, the following task and initial blocks were added - initial block to push a byte from the DataIn into the queue for further verification, also uses provided task to generate FCS bytes (see Appendix A, line 614-634); an initial block that runs state machine for receiving serial data on TX or waiting for the TX data (Appendix A, line 211-282); initial block that randomly generates three abort flags while in the receiving state after 2 bytes of data were transmitted (Appendix A, line 477-520); the task Transmit takes number of bytes to be transmitted, generates random bytes, puts them into the buffer and triggers transmission (Appendix A, line 424-461). The blocks run in parallel simulating transmission behavior while assertion are inserted in the corresponding part of the code in order to verify the behavior. Some assertion from the part A overlap with the specifications given in the part B. Such cases are further explained and referenced.

2.1 Part A

2.1.1 VerifyAbortReceive

In order to verify abort receive several bits in Rx_Sc register should be checked. For that the data is read from the address where Rx_Sc register is located. This register is an eight-bit status control register that has four read only (RO) bits - Rx_Ready, Rx_FrameError, Rx_AbortSignal, Rx_Overflow. Those bits are compared against the mask that checks corresponding bits for the abort receive. Specifically, in order to verify abort receive Rx_AbortSignal bit should be one, while others should be zero.
In order to verify Rx data buffer is zero after the abort, the RX data buffer (eight-bit register) is read into ReadData and asserted to be zero.

```
task VerifyAbortReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;
  ReadAddress(3'h2, ReadData);
  assert ((ReadData & MASK_RX_ABORT) != 0)
    $display("PASS: %s", MESSAGE_RX_ABORT);
    else $error("FAIL: %s, Got %b", MESSAGE_RX_ABORT, ReadData);

  ReadAddress(3'h3, ReadData);
  assert (ReadData === 8'h00)
      $display("PASS. Rx_Buff has correct data");
      else $error("Rx data buffer is not zero after abort: Got %h", ReadData);
endtask
```

2.1.2 VerifyNormalReceive

In order to verify normal receive, the task ReadAddress is used to read the register Rx_Sc. Then corresponding bits are checked to verify the normal receive. This is almost the same method as described above, but we use the different mask, specifically for the normal receive, Rx_Ready should be one, while other RO bits - zero.

Additionally in order to verify that the data was received correctly the Rx_Buff should be read in the same way as above. Since it is eight-bit register we are using a loop to iterate every time the data is ready and compare it to the corresponding data stored in the Data array. The Data array is a 2d array generated in the Receive task using $urandom, the same data is streamed on Rx line, so when the Rx_Ready bit is set we would expect to receive the same data we generated.

```
task VerifyNormalReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;
  wait(uin_hdlc.Rx_Ready);
  ReadAddress(3'h2, ReadData);
  assert ((ReadData & MASK_RX_NORMAL) != 0)
  $display("PASS: %s", MESSAGE_RX_NORMAL);
    else $error("FAIL: %s, Got %b", MESSAGE_RX_NORMAL, ReadData);

  for (int i = 0; i < Size; i++) begin
    wait(uin_hdlc.Rx_Ready); //Check if needed
    ReadAddress(3'h3, ReadData);
    assert (ReadData === data[i])
      $display("PASS. Rx_Buff has correct data");
      else $error("Data mismatch at index %0d: Expected %h, Got %h", i,
      ↪  data[i], ReadData);
  end
endtask
```

2.1.3 VerifyOverflowReceive

In order to verify the overflow receive behavior corresponding flags are asserted in the Rx status control register. Specifically for the overflow Rx_Ready and Rx_Overflow bits should be one, while other bits should be zero. The precedure is identical to the tasks described above.
In order to assert that the bugger contains correct data the same precedure as for the previous specification is used.

```
task VerifyOverflowReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;
  wait(uin_hdlc.Rx_Ready);
  ReadAddress(3'h2, ReadData);
  assert ((ReadData & MASK_RX_OVERFLOW) != 0)
    $display("PASS: %s", MESSAGE_RX_OVERFLOW);
    else $error("FAIL: %s, Got %b", MESSAGE_RX_OVERFLOW, ReadData);

  for (int i = 0; i < Size; i++) begin
    wait(uin_hdlc.Rx_Ready); //Check if needed
    ReadAddress(3'h3, ReadData);
    assert (ReadData === data[i])
      $display("PASS. Rx_Buff has correct data");
      else $error("Data mismatch at index %0d: Expected %h, Got %h", i,
      ↪  data[i], ReadData);
  end
endtask
```

2.2 Part B

2.2.1 Specification 1

This specification states: *Correct data in RX buffer according to RX input. The buffer should contain up to 128 bytes (this includes the 2 FCS bytes, but not the flags).*
For the following specification, correct data in the RX buffer should be asserted. This assertion is based on the received frame. Specifically, buffer should contain zeros when the frame is aborted, dropped, received extra bit. It should contain a streamed data during normal or overflow receives. Technically, this behavior was tested in the part A (abort, overflow, normal) since tasks we defined above verify the content of the buffer. Below (Specification 2) asserts correct buffer data for the non byte aligned data and dropped frame. In order to imitate the beahvior for the non byte aligned frame, in the Receive task one extra bit is added to the Rx transmission. More detailed explanation is given below.

2.2.2 Specification 2

This specification states: *Attempting to read RX buffer after aborted frame, frame error or dropped frame should result in zeros.*
For the specification 2, correct data should be asserted in the RX frame after error, abort or error frames. In order to verify this behavior we added two more tasks - VerifyDrop and VerifyNon-ByteAligned. They alongside VerifyAbortReceive (code is given in the section 2.1.1) check the buffer content after mentioned event occur. Verify drop first modifies RX status control register by writing one to the Rx_Drop flag. After one clock cycle, we read the content of the buffer and verify that it resulted in zero. Verify non byte aligned task verifies the buffer in the similar way, but the received error is checked in the Receive task itself. The behavior of the abort is similar and is explained in more details above.

```
task VerifyDrop(logic [127:0][7:0] data, int Size);
logic [7:0] ReadData;
WriteAddress(2, 8'b0000_0010); //drop the frame
@(posedge uin_hdlc.Clk);
ReadAddress(3'h3, ReadData); //read RX buffer
assert (ReadData === 8'h00) begin
  $display("PASS. Rx_Buff has correct data when drop");
end else  begin
  $error("Rx data buffer is not zero after drop: Got %h", ReadData);
end
endtask

task VerifyNonByteAligned(logic [127:0][7:0] data, int Size);
logic [7:0] ReadData;

ReadAddress(2, ReadData);
assert (ReadData ==? 8'bxxxx_x1xx) //Rx_FrameError bit
      $display("PASS: Rx status control register contains correct Frame Error
      ↪  Bit");
else begin
    $error("FAIL: Frame error didn't resulted in frame error");
end

ReadAddress(3, ReadData);
  assert (ReadData == 8'h00) begin
  $display("PASS: Rx_Buff has correct data when non byte aligned");
end
  else begin
```

```
    $error("FAIL: after non byte align we dont have 0s");
end
endtask
```

2.2.3 Specification 3

This specification states: *Correct bits set in RX status/control register after receiving frame. Remember to check all bits. I.e. after an abort the Rx Overflow bit*
The correct behavior after received frame is checked by reading the RX status control register. This specification is achieved by tasks mentioned in the section above as they read the data from the RX_Sc register on each event (abort, normal, non byte aligned, and overflow) and check the corresponding bits using masks - MASK_RX_ABORT, MASK_RX_NORMAL, MASK_RX_OVERFLOW. None byte aligned directly checks 3rd bit in the RX_Sc register that corresponds to Rx_FrameError.

2.2.4 Specification 4

This specification states: *Correct TX output according to written TX buffer.*
The correct TX output according to the written TX buffer is verified in the initial block mentioned in the section description. The block is responsible for the state machine logic. The TX output is verified during the normal data flow in the receiving state. After transmit is enabled in the Tx_Sc register and until the TX buffer is not empty a byte is collected from the serial TX line (the logic for zero removal is also part of this block) and compared to the expected data from the buffer. For more details see Apendix X, line x-x.

```
assert_rx_data: assert (my_curr == expected_data) $display("PASS: Correct TX
↪  output according to written TX buffer.");
else $error("%t: TX Monitor: Expecting 0x%02x instead of 0x%02x", $time,
↪  expected_data, my_curr);
```

2.2.5 Specification 5 and 17

This specification states: *Start and end of frame pattern generation (Start and end flag: 0111 1110).*
This specification states: *Tx Done should be asserted when the entire TX buffer has been read for transmission.*
The start and end frames are generated on the Tx line first when transmission is enabled in the Tx_Sc register and after the frame was transmitted from the Tx buffer. This design property can be asserted in the immediate assertions. When the current state is WAITING_FOR_FLAG and the STARTEND_FLAG pattern (8'b0111_1110) is detected, the state machine behaves as intended. The assertion for the start flag is implicit and is always true when the if block is entered. It can be verified anytime the process moves from the waiting state to the receiving.
The end of frame flag is also asserted implicitly when the STARTEND_FLAG is detected during the receiving state. Furthermore, this block allows us to assert Tx_Done signal (Specification 17) when the whole frame has been read, since the end of frame is reached when end signal is asserted.

```
if (my_curr == STARTEND_FLAG) begin
    assert_start_flag: assert(1); //START OF FRAME BEHAVIOR
    state = RECEIVING;
    $display("%t: TX Monitor: Going to state %s", $time, state.name());
    my_curr_size = 0;
    my_curr = 0;
end

else if (!removed_a_zero && my_curr == STARTEND_FLAG) begin
```

```
    assert_end_flag: assert(1); //5. END OF FRAME BEHAVIOR
    state = WAITING_FOR_FLAG;
    my_curr_size = 0;
    //17. Tx Done should be asserted when the entire TX buffer has been read for
    ↪  transmission.
    assert_tx_done : assert(uin_hdlc.Tx_Done) begin
      $display("PASS. Tx_done asserted after tranmission is done");
end
```

2.2.6 Specification 6

This specification states: *Zero insertion and removal for transparent transmission.*
Zero Removal logic is asserted every time zero needs to be removed from the streamed data on the TX. Below is presented the code for the bit-stuffing detector. The logic checks for the pattern of five ones which implies that the next bit could be zero that needs to be removed. Once the pattern is detected, zero is removed. This assertion is also implicit since every time the if block is entered zero is being removed.

```
if (!removed_a_zero && my_curr ==? 8'b011111xx) begin
  assert_zero_removing : assert(1)
    $display("%t: TX Monitor: Removing a zero", $time);
  my_curr_size--;
  my_curr <<= 1;
  removed_a_zero = 5;
end else begin
  removed_a_zero = removed_a_zero ? removed_a_zero-1 : 0;
end
```

2.2.7 Specification 7

This specification states: *Idle pattern generation and checking (1111 1111 when not operating).*
The idle pattern on the Tx line is high bits. It can be asserted when the state machine is in the WAITING_FOR_FLAG state. Below current size is being checked since that is an idle state size, when transmission starts it goes to the count one.

```
end else if (my_curr_size == 8) begin
    // Assumes we can have one zero in buffer in case of the start flag
    assert_idle: assert($countones(my_curr) >= 7) else $fatal();
end
```

2.2.8 Specification 8 and 9

This specification states: *Abort pattern generation and checking (1111 1110). Remember that the 0 must be sent first.*
This specification states: *When aborting frame during transmission, Tx AbortedTrans should be asserted.*
Abort pattern is asserted inside the receiving state. my_curr hold the current byte of the data being transmitted. So abort pattern is being verified whenever ABORT_FLAG is detected and it is not generated due to removed zero (since the transmitted data can accidentaly match the abort flag pattern).
Furthermore, when the frame is aborted as per specification 9, the signal Tx_AbortedTrans should also be asserted. That is done in the same if block by directly checking the signal.

```
if (!removed_a_zero && my_curr == ABORT_FLAG) begin // ABORT BEHAVIOR
    //8. Abort pattern generation and checking (1111 1110). Remember that the 0
    ↪ must be sent first
    assert(1) $display("PASS: Abort pattern 0x%02x received after abort set in
    ↪ TX_SC reg", my_curr);
      else $error("FAIL. Expected 0xfe while received %x", my_curr); //roughly
      ↪ impossible if we are in this IF-statemnt
    //9. When aborting frame during transmission, Tx AbortedTrans should be
    ↪ asserted.
    assert_abort_flag: assert(uin_hdlc.Tx_AbortedTrans == 1) $display("PASS: Tx
    ↪ AbortedTrans is set correctly after abort");
      else $error("%t: TX Monitor: Current 0x%02x (size: %0d) (removed a zero:
      ↪ %0d) (Tx: %0d) ", $time, my_curr, my_curr_size, removed_a_zero,
      ↪ uin_hdlc.Tx);
    state = WAITING_FOR_FLAG;
    my_curr_size = 0;
    $display("%t: TX Monitor: Going to state %s", $time, state.name());
end
```

However, the same behavior can also be asserted by checking an actual TX status control register. The code below is a snippet from the initial block that is responsible for the abort signal generation. Below we can see that the assertion verifies that the bits read from the Tx_Sc register match flags for the aborted transmission and done transmission.

```
assert (ReadData == 8'b0000_1001) begin
  $display("PASS: AbortedTrans in TX_SC (status control) register is correct");
end else begin
  $error("FAIL: Expected Tx_sc = 0x09, Received Tx_sc = 0x%h", ReadData);
end
```

2.2.9 Specification 11

This specification states: *CRC generation and checking.*
The CRC is generated in the initial block when transmission is enabled. It is generated using provided task GenerateFCSBytes that takes the data from the buffer to generate CRC. Since we have two bytes of the CRC we shift out a byte in order to access the next CRC byte. It is checked every time the buffer is empty (no more data is available) and control status register still holds the flag that the transmission is enabled.

```
if (my_data_q.size() == 0 && reg_tx_sc.enable) begin
    //11. CRC generation and Checking.
    assert_tx_fcs: assert (my_curr == my_fcs[7:0])
    else $error("%t: TX Monitor: Expecting 0x%02x instead of 0x%02x (FCS)",
    ↪ $time, my_fcs[7:0], my_curr);
    my_fcs >>= 8;
end
```

2.2.10 Specification 18

This specification states: *Tx Full should be asserted after writing 126 or more bytes to the TX buffer (overflow).*
This specification is verified inside the Transmit task. Specifically when the number of bytes to be transmitted is equal or over 126, we are reading the status control register and asserting the flag that the buffer is full.

```
if (num_bytes_sent_to_tx_buffer >= 126) begin
  ReadAddress(0, tx_statusControl);
  tx_full_assert : assert(tx_statusControl.full) begin
    $display("PASS. TX Full asserted after receiving more or 126 bytes");
  end else begin
    $error("FAIL. num_bytes_sent_to_tx_buffer is %0d Tx full in SC reg is %0d",
    ↪  num_bytes_sent_to_tx_buffer, tx_statusControl.full);
  end
end
```

3 Concurrent assertions

SystemVerilog concurrent assertions are a formal verification construct used to specify temporal properties of digital designs over time. They are evaluated over simulation time and allow to check it the sequences of events and signal values satisfy given conditions. Unlike immediate assertions (which check conditions at a single point in time), concurrent assertions describe temporal behaviors using sequence expressions and are evaluated concurrently with the simulation Bergeron 2006. They are especially useful for verifying complex protocols, timing relationships, and state transitions, and play a key role in simulation-based and formal verification methodologies.

Many of the properties in this project (for example, RX_FlagDetect, RX_AbortSignalWhenValid, RX_Overflow_Detect) involve verifying sequences of events or conditions that evolve over multiple clock cycles. Concurrent assertions are better suited for this because they can express temporal relationships like "if X happens, then Y must happen after N cycles."

3.1 Part A

3.1.1 Specification RX_FlagDetect

This specification states: *Write the Rx flag sequence, which identifies a flag.* This assertion checks whether a specific flag sequence is detected correctly in the Rx signal and ensures that the Rx_FlagDetect signal is asserted two clock cycles after the flag sequence is received. The sequence Rx_flag defines the expected flag pattern in the Rx signal: !Rx #1 means the sequence starts with a low signal (0), Rx [*6] ##1 followed by six consecutive high signals (111111), !Rx ends with another low signal (0).
The property uses the Rx_flag sequence to verify the behavior of the Rx_FlagDetect signal: if the Rx_flag sequence is detected, the Rx_FlagDetect signal must be asserted exactly two clock cycles after.

```
  sequence Rx_flag;
    // INSERT CODE HERE
    @(posedge Clk)
      !Rx ##1  // 0
      Rx [*6] ##1 // 111111
      !Rx;      // 0
  endsequence

  // Check if flag sequence is detected
  property RX_FlagDetect;
    @(posedge Clk) disable iff (!Rst)
    (Rx_flag) |-> ##2 Rx_FlagDetect;
  endproperty

  RX_FlagDetect_Assert : assert property (RX_FlagDetect) begin
```

```
  $display("PASS: Flag detect");
end else begin
  $error("Flag sequence did not generate FlagDetect");
  ErrCntAssertions++;
end
```

3.1.2 Specification Rx_AbortSignal

This specification states: *Write the Rx AbortSignal property, which verifies that the abort signal is raised when an abort pattern is observed during a valid frame.* This specification is the same as Specification 10 (Part B), and covered in subsection 3.2.1.

3.2 Part B

3.2.1 Specification 10

This specification states: *Abort pattern detected during valid frame should generate Rx AbortSignal.* It has been decided to write this assertion as a concurrent one because it evaluates the internal signals Rx_AbortDetect, Rx_ValidFrame and Rx_AbortSignal, and the relationship between them. This assertion is verifying a protocol-level behavior where an abort signal Rx_AbortSignal must be generated in response to an abort detection Rx_AbortDetect during a valid frame.
If Rx_AbortDetect and Rx_ValidFrame are both high in a given clock cycle, then Rx_AbortSignal must be asserted (high) in the next clock cycle. The assertion ensures that the system correctly generates the Rx_AbortSignal one cycle after detecting an abort pattern during a valid frame. If Rx_AbortDetect and Rx_ValidFrame are both high in one cycle, but Rx_AbortSignal is not asserted in the next cycle, the assertion will fail. This would indicate a bug in the design, such as a missing or delayed generation of the Rx_AbortSignal.

```
property RX_AbortSignalWhenValid;
  @(posedge Clk) disable iff (!Rst)
  (Rx_AbortDetect && Rx_ValidFrame) |=> Rx_AbortSignal;
endproperty

RX_AbortSignalWhenValid_Assert : assert property (RX_AbortSignalWhenValid)
begin
  $display("PASS. AbortSignal was generated when abort pattern was detected");
end else begin
  $error("AbortSignal did not go high after AbortDetect during validframe");
  ErrCntAssertions++;
end
```

3.2.2 Specification 13

This specification states: *When receiving more than 128 bytes, Rx Overflow should be asserted.* This specification it is choosen to be written as a concurrent assertion becouse it verifies a temporal property that develops over multible clk cycles. It monitores the reception of 129 bytes and ensures the overflow signal is asserted.
The property monitors a sequence of events over time: the start of a valid frame $rose(Rx_ValidFrame), the reception of 129 bytes $rose(Rx_NewByte) [→ 129], and the assertion of Rx_Overflow. This temporal relationship suited to be expressed with an immediate assertion.

```
property RX_Overflow_Detect;
  @(posedge Clk) disable iff (!Rst || !Rx_ValidFrame)
  $rose(Rx_ValidFrame) ##0 ($rose(Rx_NewByte) [->129]) |=> $rose(Rx_Overflow)
```

```
endproperty

RX_Overflow_Assert : assert property (RX_Overflow_Detect) begin
  $display("PASS: RX_Overflow detected afrer more than 128 bytes received");
end else begin
  $error("RX_Overflow did not go high after receiving more than 128 bytes");
  ErrCntAssertions++;
end
```

3.2.3 Specification 14

The specification states: *Rx FrameSize should equal the number of bytes received in a frame (max. 126 bytes = 128 bytes in buffer – 2 FCS bytes).* for this specification it is decided to write an always_ff logic block for the byte counting. The counter byte_counter is used to track the number of bytes received in a frame. The byte_counter is reset to 0 when Rst is deasserted or when the end of a frame Rx_EoF is detected. The counter increments by 1 on every rising edge of Rx_NewByte, which indicates a new byte has been received. Regarding the property, on the antecedent side it checks for the rising edge of Rx_EoF $rose(Rx_EoF), which indicates the end of a frame. It also ensures that there are no frame errors !Rx_FrameError and no overflow !Rx_Overflow. After the antecedent is satisfied, on the consequet side the property asserts that the Rx_FrameSize signal matches the exact number of bytes received in the frame, which is byte_counter - 2 (the subtraction accounts for FCS bytes).

```
logic [7:0] byte_counter; // Counter to track the number of bytes received
// Update the byte counter
always_ff @(posedge Clk or negedge Rst) begin
  if (!Rst) begin
    byte_counter <= 8'd0;
  end else if ($fell(Rx_EoF)) begin
    byte_counter <= 8'd0; // Reset the counter when not in a valid frame
  end else if ($rose(Rx_NewByte)) begin
    byte_counter <= byte_counter + 1; // Increment the counter on each new
    ↪  byte
    $display("byte_counter %0d", byte_counter);
  end
end

property RX_FrameSize_Exact;
  @(posedge Clk) disable iff (!Rst)
  ($rose(Rx_EoF) and !Rx_FrameError and !Rx_Overflow) |-> ##1 (Rx_FrameSize ==
  ↪  (byte_counter-2));
endproperty

RX_FrameSize_Exact_Assert : assert property (RX_FrameSize_Exact) begin
  $display("PASS: Rx_FrameSize matches the exact number of bytes received.");
end else begin
  $error("FAIL: Rx_FrameSize does not match the exact number of bytes received.
  ↪  Expected: %0d, Got: %0d", byte_counter-2, Rx_FrameSize);
  ErrCntAssertions++;
end
```

3.2.4 Specification 15

The specification states: *Rx Ready should indicate byte(s) in RX buffer is ready to be read.*
It is convinient to express this specification as an concurrent assertion becouse it describes a behaiviour over multiple cycles: the transition of Rx_Ready from low to high and the subsequent

assertion of Rx_EoF. The protocol requires that the RX buffer readiness Rx_Ready is correctly synchronized with the end-of-frame signal Rx_EoF. The property verifies that transition of Rx_Ready from low to high is followed by the assertion of Rx_EoF.

```
property RX_Ready;
  @(posedge Clk) disable iff (!Rst)
  (!Rx_Ready ##1 Rx_Ready) |-> (Rx_EoF); //Whole RX frame has been received,
  ↪  end of frame
endproperty

RX_Ready_Assert : assert property(RX_Ready) begin
  $display("PASS: Bytes in RX Buffer are ready to be read");
end else begin
  $error("FAIL: EoF is not asserted when buffer is ready");
  ErrCntAssertions++;
end
```

3.2.5 Specification 16

The specification states: *Non-byte aligned data or error in FCS checking should result in frame error.* This specification is verified as a concurrent assertion because it involves temporal relationships. The property checks the temporal relationship between Rx_FCSerr, Rx_FCSen, and Rx_FrameError: Rx_FCSerr must transition from 0 to 1 within one clock cycle. Rx_FrameError must be asserted as a result of this transition. The HDLC module uses the FCS to detect errors in the received frame. If the FCS check fails (Rx_FCSerr becomes 1), it indicates that the frame is corrupted. When an FCS error is detected and FCS error detection is enabled, the module should assert Rx_FrameError to indicate that the frame is invalid. This property ensures that the module correctly flags a frame error Rx_FrameError in response to an FCS error.

```
property FCSerrFrameerr;
  @(posedge Clk) disable iff (!Rst)
  !Rx_FCSerr ##1 Rx_FCSerr && Rx_FCSen |=> Rx_FrameError;
endproperty

FCSerrFrameerr_Assert : assert property(FCSerrFrameerr) begin
  $display("PASS. Non-byte aligned data or error in FCS checking resulted in
  ↪  frame error");
end else begin
  $error("FAIL. FCSerrFrameerr_Assertion does not work");
  ErrCntAssertions++;
end
```

3.2.6 Specification 17

The specification states: *Tx Done should be asserted when the entire TX buffer has been read for transmission.* The purpose of this property and assertion is to verify the behavior of the HDLC module when the TX buffer completes its transmission. The property checks the temporal relationship between Tx_DataAvail and Tx_Done: Tx_Done must be asserted immediately after Tx_DataAvail transitions from 1 to 0. The assertion ensures that the module properly handles the transition of the TX buffer from having data available to being empty.

```
property TX_Done;
  @(posedge Clk) disable iff (!Rst)
  Tx_DataAvail ##1 !Tx_DataAvail |-> Tx_Done;
endproperty
```

```
TX_Done_Assert : assert property(TX_Done) begin
  $display("PASS: Imm. Tx_done asserted after no data available");
end else begin
  $error("FAIL: Imm Tx_done not asserted properly after no data available");
  ErrCntAssertions++;
end
```

4 Discussions

The provided testbench for the HDLC protocol implementation show a good approach to verifying both RX and TX functionalities through a combination of immediate assertions, tasks, and simulation logic. Immediate assertions in testPr_hdlc.sv focus on verifying specific behaviors, such as making sure the RX status/control register have the correct state after events like normal frame reception, buffer overflow, or aborts. These assertions provide detailed feedback, making it easier to identify and debug issues. Nevertheless, there are potential corner cases that may not be fully addressed. For example, they do not clearly handle cases where multiple events occur at the same time, such as an abort during an overflow. The testbench does not include the stress testing. For example it does not check for high frequency changes or some edge cases for the RX and TX signals. The concurrent assertions in assertions_hdlc.sv complement the immediate assertions: they check time-critical behaviors by ensuring signals like Rx_FlagDetect are asserted on the specific clock. These assertions though could also benefit from extra coverage for the edge cases, for example, fir the overlapping flag sequences or for the incomplete sequences caused by the signal glitches. Our testbench includes tasks like VerifyAbortReceive and VerifyOverflowReceive to assert the correct data in the RX buffer, but it does not explicitly check the behavior of the buffer under the reset or after simultaneous events. Overall, the testbench provides good foundation for functional verification but could be improved by adding more corner cases and expanding its coverage.

5 Coverage

The coverage report was created for the given HDLC design (see Appendix A, line 636-839). The covergroup consists of the coverpoints for the internal and external signals for the TX and RX. It takes all signals presented in the rtl folder in hdlc.sv file. Provided stimuli covered the total of 91.97%. This result can be further improved by executing Trasmit and Receive more times. Currently we have total of 49 coverpoints and 395 coverbins (356 covered). Some edge cases might not be covered due to lack of requested specifications and/or assertion type choice. For example, specification 14 asserts that RX frame size should be equal to the number of bytes received in a frame. The way we approached this is by using always block to count received bytes and compare that number to the Rx_FrameSize (internal signal). However, this could also be verified in the immediate assertions, by comparing the number of bytes we sent to the number of bytes stored in the RX frame length (Rx_Len) register. Therefore, specific choice of how the assertion is verified impacts the coverage.

6 Conclusion

In conclusion, this paper provides detailed description of how specifications were asserted for the HDLC module. It covered both part A and part B of the project and explained the overlap between requirements. In order to make clear our choice of using immediate or concurrent assertions, each section has a small explanation in the beginning. The report follows each specification and explains how the assertion is implemented and how it is integrated in the initial blocks (for concurrent assertions).

Bibliography

Bergeron, Janick (2006). *Writing Testbenches Using SystemVerilog*. Springer Science & Business Media.

Guide, Verification (2025). *SystemVerilog Assertions*. URL: https://verificationguide.com/systemverilog/systemverilog-assertions (visited on 16th Apr. 2025).

Appendix

A testPr_hdlc.sv

```
1  ////////////////////////////////////////////////
2  // Title:   testPr_hdlc
3  // Author:
4  // Date:
5  ////////////////////////////////////////////////
6
7  /* testPr_hdlc contains the simulation and immediate assertion code of the
8     testbench.
9
10    For this exercise you will write immediate assertions for the Rx module which
11    should verify correct values in some of the Rx registers for:
12    - Normal behavior
13    - Buffer overflow
14    - Aborts  initial forever begin
15
16
17    HINT:
18    - A ReadAddress() task is provided, and addresses are documentet in the
19      HDLC Module Design Description
20 */
21
22 program testPr_hdlc(
23   in_hdlc uin_hdlc
24 );
25
26   int TbErrorCnt;
27
28   /*****************************************************************************
29    *                                                                           *
30    *                              Student code                                 *
31    *                                                                           *
32    *****************************************************************************/
33
34   // Define bit masks for each field
35   localparam logic [7:0] MASK_RX_ABORT        = 8'b0000_1000;
36   const string MESSAGE_RX_ABORT =     "RX Status/Control received abort
     ↪  sequence";
37
38   localparam logic [7:0] MASK_RX_NORMAL       = 8'b0000_0001;
39   const string MESSAGE_RX_NORMAL =    "RX Status/Control received normal
     ↪  sequence";
40
41   localparam logic [7:0] MASK_RX_OVERFLOW     = 8'b0001_0001;
42   const string MESSAGE_RX_OVERFLOW =  "RX Status/Control received overflow
     ↪  sequence";
43
44   //3. Correct bits set in RX status/control register after receiving frame.
45   //Remember to check all bits. I.e. after an abort the Rx Overflow bit should be
     ↪  0, unless an overflow also occurred.
46
47   // VerifyAbortReceive should verify correct value in the Rx status/control
48   // register, and that the Rx data buffer is zero after abort.
49   task VerifyAbortReceive(logic [127:0][7:0] data, int Size);
```

```
50      logic [7:0] ReadData;
51
52      // Read the RX status/control register at address 0x2
53      ReadAddress(3'h2, ReadData);
54
55      // Assert the mask in the RX status/control register
56      assert ((ReadData & MASK_RX_ABORT) != 0)
57        $display("PASS: %s", MESSAGE_RX_ABORT);
58        else $error("FAIL: %s, Got %b", MESSAGE_RX_ABORT, ReadData);
59
60      ReadAddress(3'h3, ReadData);
61        assert (ReadData === 8'h00)
62          $display("PASS. Rx_Buff has correct data when abort");
63          else $error("Rx data buffer is not zero after abort: Got %h", ReadData);
64
65    endtask
66
67    // VerifyNormalReceive should verify correct value in the Rx status/control
68    // register, and that the Rx data buffer contains correct data.
69    task VerifyNormalReceive(logic [127:0][7:0] data, int Size);
70      logic [7:0] ReadData;
71      wait(uin_hdlc.Rx_Ready);
72
73      // Read the RX status/control register at address 0x2
74      ReadAddress(3'h2, ReadData);
75
76      // Assert the mask in the RX status/control register
77      assert ((ReadData & MASK_RX_NORMAL) != 0)
78        $display("PASS: %s", MESSAGE_RX_NORMAL);
79        else $error("FAIL: %s, Got %b", MESSAGE_RX_NORMAL, ReadData);
80
81      // Loop through "Size" and compare "data" with "ReadData"
82      for (int i = 0; i < Size; i++) begin
83        wait(uin_hdlc.Rx_Ready); //Check if needed
84        ReadAddress(3'h3, ReadData);
85        assert (ReadData === data[i])
86          $display("PASS. Rx_Buff has correct data when normal");
87          else $error("Data mismatch at index %0d: Expected %h, Got %h", i,
            ↪  data[i], ReadData);
88      end
89
90    endtask
91
92    // VerifyNormalReceive should verify correct value in the Rx status/control
93    // register, and that the Rx data buffer contains correct data.
94    task VerifyOverflowReceive(logic [127:0][7:0] data, int Size);
95      logic [7:0] ReadData;
96      wait(uin_hdlc.Rx_Ready);
97
98      // Read the RX status/control register at address 0x2
99      ReadAddress(3'h2, ReadData);
100
101     // Assert the mask in the RX status/control register
102     assert ((ReadData & MASK_RX_OVERFLOW) != 0)
103       $display("PASS: %s", MESSAGE_RX_OVERFLOW);
104       else $error("FAIL: %s, Got %b", MESSAGE_RX_OVERFLOW, ReadData);
105
106     // Loop through "Size" and compare "data" with "ReadData"
```

```
107       for (int i = 0; i < Size; i++) begin
108         wait(uin_hdlc.Rx_Ready); //Check if needed
109         ReadAddress(3'h3, ReadData);
110         assert (ReadData === data[i])
111           $display("PASS. Rx_Buff has correct data when overflow");
112           else $error("Data mismatch at index %0d: Expected %h, Got %h", i,
              ↪  data[i], ReadData);
113       end
114
115     endtask
116
117     //2. Attempting to read RX buffer after frame error or dropped frame should
        ↪  result in zeros.
118     task VerifyDrop(logic [127:0][7:0] data, int Size);
119       logic [7:0] ReadData;
120       WriteAddress(2, 8'b0000_0010); //drop the frame
121       @(posedge uin_hdlc.Clk);
122       ReadAddress(3'h3, ReadData); //read RX buffer
123       assert (ReadData === 8'h00) begin
124         $display("PASS. Rx_Buff has correct data when drop");
125       end else  begin
126         $error("Rx data buffer is not zero after drop: Got %h", ReadData);
127       end
128     endtask
129
130     task VerifyNonByteAligned(logic [127:0][7:0] data, int Size);
131       logic [7:0] ReadData;
132
133       ReadAddress(2, ReadData);
134       assert (ReadData ==? 8'bxxxx_x1xx) //Rx_FrameError bit
135             $display("PASS: Rx status control register contains correct Frame Error
              ↪  Bit");
136       else begin
137           $error("FAIL: Frame error didn't resulted in frame error");
138       end
139
140       ReadAddress(3, ReadData);
141             assert (ReadData == 8'h00) begin
142         $display("PASS: Rx_Buff has correct data when non byte aligned");
143       end
144             else begin
145               $error("FAIL: after non byte align we dont have 0s");
146       end
147     endtask
148
149     /*****************************************************************************
150      *                                                                           *
151      *                              Simulation code                              *
152      *                                                                           *
153      *****************************************************************************/
154
155     typedef enum {
156       ADDR_TX_CS   = 0,
157       ADDR_TX_BUFF = 1
158     } hdlc_reg_addrs_e;
159
160     typedef struct packed{
161       bit [7:5] reserved;
```

```
162     bit full;
163     bit abortedTrans;
164     bit abortFrame;
165     bit enable;
166     bit done;
167   } reg_tx_sc_t;
168
169   reg_tx_sc_t reg_tx_sc;
170
171   byte my_data_q[$];
172   byte my_curr;
173   shortint my_fcs;
174   byte unsigned my_curr_size;
175   const byte STARTEND_FLAG = 8'h7E;
176   const byte ABORT_FLAG = 8'hFE;
177   //const byte my_flag = 8'hBF;
178   byte removed_a_zero;
179
180   enum {
181     WAITING_FOR_FLAG,
182     RECEIVING
183   } state;
184
185   initial begin
186     state = WAITING_FOR_FLAG;
187     my_curr = '1;
188     my_curr_size = 0;
189     removed_a_zero = 0;
190   end
191
192   initial forever begin
193     @(negedge uin_hdlc.Clk)
194     if (uin_hdlc.WriteEnable && uin_hdlc.Address == ADDR_TX_CS) begin
195       reg_tx_sc = uin_hdlc.DataIn[7:0];
196       if (reg_tx_sc.enable) begin
197         automatic logic [127:0] [7:0] data = 0;
198         foreach (my_data_q[idx]) begin
199           data[idx] = my_data_q[idx];
200         end
201         GenerateFCSBytes(data, my_data_q.size(), my_fcs);
202         $display("%t: Register Monitor: Detected start of TX transmission.
          ↪  Expecting FCS: 0x%04x", $time, my_fcs);
203       end
204     end
205     if (uin_hdlc.WriteEnable && uin_hdlc.Address == ADDR_TX_BUFF) begin
206       my_data_q.push_back(uin_hdlc.DataIn[7:0]);
207       $display("%t: Register Monitor: Pushed 0x%02x to data queue", $time,
        ↪  uin_hdlc.DataIn[7:0]);
208     end
209   end
210
211   initial forever begin
212     @(posedge uin_hdlc.Clk);
213     #0;
214     my_curr >>= 1;
215     my_curr[7] |= uin_hdlc.Tx;
216     if (my_curr_size < 8) my_curr_size ++;
```

```
217     if (my_curr != 8'hff) $display("%t: TX Monitor: Current 0x%02x (size: %0d)
        ↪  (removed a zero: %0d) (Tx: %0d) ", $time, my_curr, my_curr_size,
        ↪  removed_a_zero, uin_hdlc.Tx);
218     if (state == WAITING_FOR_FLAG) begin
219       //$display("INSIDE THE WAITING_FOR_FLAG");
220       if (my_curr == STARTEND_FLAG) begin
221         assert_start_flag: assert(1); //5. implicitly asserted
222         state = RECEIVING;
223         $display("%t: TX Monitor: Going to state %s", $time, state.name());
224         my_curr_size = 0;
225         my_curr = 0;
226       end else if (my_curr_size == 8) begin
227         // Assumes we can have one zero in buffer in case of the start flag
228         //7. Idle pattern generation and checking (1111 1111 when not
            ↪  operating).
229         assert_idle: assert($countones(my_curr) >= 7) else $fatal();
230       end
231     end else if (state == RECEIVING) begin
232       if (!removed_a_zero && my_curr == ABORT_FLAG) begin // ABORT BEHAVIOR
233         //8. Abort pattern generation and checking (1111 1110). Remember that the
            ↪  0 must be sent first
234         assert(1) $display("PASS: Abort pattern 0x%02x received after abort set
            ↪  in TX_SC reg", my_curr);
235           else $error("FAIL. Expected 0xfe while received %x", my_curr);
              ↪  //roughly impossible if we are in this IF-statemnt
236         //9. When aborting frame during transmission, Tx AbortedTrans should be
            ↪  asserted.
237         assert_abort_flag: assert(uin_hdlc.Tx_AbortedTrans == 1) $display("PASS:
            ↪  Tx AbortedTrans is set correctly after abort");
238           else $error("%t: TX Monitor: Current 0x%02x (size: %0d) (removed a
              ↪  zero: %0d) (Tx: %0d) ", $time, my_curr, my_curr_size,
              ↪  removed_a_zero, uin_hdlc.Tx);
239         state = WAITING_FOR_FLAG;
240         my_curr_size = 0;
241         $display("%t: TX Monitor: Going to state %s", $time, state.name());
242       end else if (!removed_a_zero && my_curr == STARTEND_FLAG) begin
243         assert_end_flag: assert(1); //5. END OF FRAME BEHAVIOR
244         state = WAITING_FOR_FLAG;
245         my_curr_size = 0;
246         //17. Tx Done should be asserted when the entire TX buffer has been read
            ↪  for transmission.
247         assert_tx_done : assert(uin_hdlc.Tx_Done) begin
248           $display("PASS. Tx_done asserted after tranmission is done");
249         end else begin
250           $error("FAIL. Tx_done is not asserted after transmission or idk");
251           //ErrCntAssertions++;
252         end
253         $display("%t: TX Monitor: Going from ENDFRAME to state %s", $time,
            ↪  state.name());
254       end else begin //NORMAL FLOW
255         if (!removed_a_zero && my_curr ==? 8'b011111xx) begin
256           assert_zero_removing : assert(1)
257             $display("%t: TX Monitor: Removing a zero", $time);
258           my_curr_size--;
259           my_curr <<= 1;
260           removed_a_zero = 5;
261         end else begin
262           removed_a_zero = removed_a_zero ? removed_a_zero-1 : 0;
```

```
263         end
264         if (my_curr_size == 8 && !uin_hdlc.Tx_AbortedTrans) begin
265           //$display("%t: TX Monitor: Received byte 0x%02x.", $time, my_curr);
266           // When out of bytes in the queue expect to receive the FCS
267           if (my_data_q.size() == 0 && reg_tx_sc.enable) begin
268             //11. CRC generation and Checking.
269             assert_tx_fcs: assert (my_curr == my_fcs[7:0])
270             else $error("%t: TX Monitor: Expecting 0x%02x instead of 0x%02x
                ↪  (FCS)", $time, my_fcs[7:0], my_curr);
271             my_fcs >>= 8;
272           end else begin
273             automatic byte expected_data = my_data_q.pop_front();
274             //4. correct TX data
275             assert_rx_data: assert (my_curr == expected_data)
276             else $error("%t: TX Monitor: Expecting 0x%02x instead of 0x%02x",
                ↪  $time, expected_data, my_curr);
277           end
278           my_curr_size = 0;
279         end
280       end
281     end
282   end
283
284   initial begin
285     $display("****************************************************************");
286     $display("%t - Starting Test Program", $time);
287     $display("****************************************************************");
288
289     Init();
290
291     //Receive: Size, Abort, FCSerr, NonByteAligned, Overflow, Drop, SkipRead
292     Receive( 10, 0, 0, 0, 0, 0, 0); //Normal
293     Receive( 40, 1, 0, 0, 0, 0, 0); //Abort
294     Receive(126, 0, 0, 0, 1, 0, 0); //Overflow
295     Receive( 45, 0, 0, 0, 0, 0, 0); //Normal
296     Receive(126, 0, 0, 0, 0, 0, 0); //Normal
297     Receive(122, 1, 0, 0, 0, 0, 0); //Abort
298     Receive(126, 0, 0, 0, 1, 0, 0); //Overflow
299     Receive( 25, 0, 0, 0, 0, 0, 0); //Normal
300     Receive( 47, 0, 0, 0, 0, 0, 0); //Normal
301     Receive( 25, 0, 0, 0, 0, 1, 0); //DROP
302     Receive( 47, 0, 0, 1, 0, 0, 0); //NonByteAligned
303
304     repeat (1) begin
305       Transmit(0);
306       //Transmit(1);
307       #1ms;
308       //Transmit(2);
309       #1ms;
310       Transmit(3);
311       #1ms;
312       Transmit(4);
313       Transmit(8);
314       Transmit(16);
315       Transmit(32);
316       Transmit(32);
317       Transmit(64);
318       Transmit(64);
```

```
319       Transmit(126);
320
321
322     end
323
324     $display("************************************************************");
325     $display("%t - Finishing Test Program", $time);
326     $display("************************************************************");
327     $stop;
328   end
329
330   final begin
331
332     $display("*********************************");
333     $display("*                               *");
334     $display("* \tAssertion Errors: %0d\t  *", TbErrorCnt +
      ↪  uin_hdlc.ErrCntAssertions);
335     $display("*                               *");
336     $display("*********************************");
337
338   end
339
340   task Init();
341     uin_hdlc.Clk         =   1'b0;
342     uin_hdlc.Rst         =   1'b0;
343     uin_hdlc.Address     = 3'b000;
344     uin_hdlc.WriteEnable =   1'b0;
345     uin_hdlc.ReadEnable  =   1'b0;
346     uin_hdlc.DataIn      =     '0;
347     uin_hdlc.TxEN        =   1'b1;
348     uin_hdlc.Rx          =   1'b1;
349     uin_hdlc.RxEN        =   1'b1;
350
351     TbErrorCnt = 0;
352
353     #1000ns;
354     uin_hdlc.Rst         =   1'b1;
355   endtask
356
357   task WriteAddress(input logic [2:0] Address ,input logic [7:0] Data);
358     @(posedge uin_hdlc.Clk);
359     uin_hdlc.Address     = Address;
360     uin_hdlc.WriteEnable = 1'b1;
361     uin_hdlc.DataIn      = Data;
362     @(posedge uin_hdlc.Clk);
363     uin_hdlc.WriteEnable = 1'b0;
364   endtask
365
366   task ReadAddress(input logic [2:0] Address ,output logic [7:0] Data);
367     @(posedge uin_hdlc.Clk);
368     uin_hdlc.Address    = Address;
369     uin_hdlc.ReadEnable = 1'b1;
370     #100ns;
371     Data                = uin_hdlc.DataOut;
372     @(posedge uin_hdlc.Clk);
373     uin_hdlc.ReadEnable = 1'b0;
374   endtask
375
```

```
376   task InsertFlagOrAbort(int flag);
377     @(posedge uin_hdlc.Clk);
378     uin_hdlc.Rx = 1'b0;
379     @(posedge uin_hdlc.Clk);
380     uin_hdlc.Rx = 1'b1;
381     @(posedge uin_hdlc.Clk);
382     uin_hdlc.Rx = 1'b1;
383     @(posedge uin_hdlc.Clk);
384     uin_hdlc.Rx = 1'b1;
385     @(posedge uin_hdlc.Clk);
386     uin_hdlc.Rx = 1'b1;
387     @(posedge uin_hdlc.Clk);
388     uin_hdlc.Rx = 1'b1;
389     @(posedge uin_hdlc.Clk);
390     uin_hdlc.Rx = 1'b1;
391     @(posedge uin_hdlc.Clk);
392     if(flag)
393       uin_hdlc.Rx = 1'b0; //end
394     else
395       uin_hdlc.Rx = 1'b1; //abort
396   endtask
397
398   task MakeRxStimulus(logic [127:0][7:0] Data, int Size);
399     logic [4:0] PrevData;
400     PrevData = '0;
401     for (int i = 0; i < Size; i++) begin
402       for (int j = 0; j < 8; j++) begin
403         if(&PrevData) begin
404           @(posedge uin_hdlc.Clk);
405           uin_hdlc.Rx = 1'b0;
406           PrevData = PrevData >> 1;
407           PrevData[4] = 1'b0;
408         end
409
410         @(posedge uin_hdlc.Clk);
411         uin_hdlc.Rx = Data[i][j];
412
413         PrevData = PrevData >> 1;
414         PrevData[4] = Data[i][j];
415       end
416     end
417   endtask
418
419   // Write Tx_Enable in Tx_SC
420   // If still more to send wait for Tx_Done
421   //
422   // What happpens when writing to Tx buffer after enable before it has its
      ↪  content
423   // What happpens when using 0 size buffer
424   task Transmit(shortint num_bytes);
425     logic  [7:0] ReadData;
426     reg_tx_sc_t tx_statusControl;
427     automatic byte num_bytes_sent_to_tx_buffer = 0;
428     //byte bytes_pushed_to_buffer[£];
429
430     $display("%t: Transmit(%0d)", $time, num_bytes);
431
432     // What are we even doing here?
```

```
433     if (!(num_bytes inside {[1:128]})) begin
434       $display("%t: Cannot send more bytes that what Tx Buffer can hold!",
          ↪  $time);
435       return;
436     end
437
438     // Clearing Status and Control register
439     tx_statusControl = '0;
440     WriteAddress(0, tx_statusControl);
441
442     // Push to buffer util Tx_Full unless all is in
443
444     $display("%t: TX transmission: Filling Tx Buffer", $time);
445     while (!tx_statusControl.full && num_bytes_sent_to_tx_buffer < num_bytes)
        ↪  begin
446       byte to_push_to_buffer;
447       to_push_to_buffer = $urandom;
448       WriteAddress(1, to_push_to_buffer);
449       num_bytes_sent_to_tx_buffer++;
450       //bytes_pushed_to_buffer.push_back(to_push_to_buffer);
451       ReadAddress(0, tx_statusControl);
452     end
453     //18. Tx Full should be asserted after writing 126 or more bytes to the TX
        ↪  buffer (overflow).
454     if (num_bytes_sent_to_tx_buffer >= 126) begin
455       ReadAddress(0, tx_statusControl);
456       tx_full_assert : assert(tx_statusControl.full) begin
457         $display("PASS. TX Full asserted after receiving more or 126 bytes");
458       end else begin
459         $error("FAIL. num_bytes_sent_to_tx_buffer is %0d Tx full in SC reg is
            ↪  %0d", num_bytes_sent_to_tx_buffer, tx_statusControl.full);
460       end
461     end
462
463     // Start sending by writing to enable in Tx_CS register
464     tx_statusControl.enable = 1;
465     WriteAddress(0, tx_statusControl);
466     $display("%t: TX transmission: Started", $time);
467
468     wait(uin_hdlc.Tx_Done);
469     $display("%t: TX transmission: TX Buffer Done", $time);
470
471     wait(state == WAITING_FOR_FLAG);
472     $display("%t: TX transmission: Done (flag detected)", $time);
473     #5us;
474   endtask
475
476 int abort_count = 0;
477   initial forever begin //SENDING ABORT
478   @(posedge uin_hdlc.Clk);
479   #0;
480     if (abort_count < 3 && //limit number of aborts
481         $urandom_range(0, 99) < 2 && // ~2% probability
482         state == RECEIVING &&
483         !(!removed_a_zero && my_curr == ABORT_FLAG) &&
484         !(!removed_a_zero && my_curr == STARTEND_FLAG)
485     ) begin //NORMAL FLOW
486       logic  [7:0] ReadData;
```

```
487         reg_tx_sc_t tx_statusControl;
488         abort_count++;
489         repeat(16) @(posedge uin_hdlc.Clk); //wait for half of the frame
490         //repeat(num_bytes/2) @(posedge uin_hdlc.Clk); //abort in the middle
491         $display("%0t: TX transmission: Sending abort after %0d number of cycles",
            ↪ $time, 16);
492         // Send the abort signal
493         tx_statusControl = '0;
494         tx_statusControl.abortFrame = 1;
495         tx_statusControl.enable = 0;
496         WriteAddress(0, tx_statusControl);
497         @(posedge uin_hdlc.Clk);
498         @(posedge uin_hdlc.Clk);
499 
500         ReadAddress(3'b000, ReadData);
501         my_data_q = {}; //empty the queue
502         ReadData = ReadData & 8'b1111_1001;
503         $display("read data from the tx_statusControl is %x", ReadData);
504         // asserts that TX status control register is set correctly
505         assert (ReadData == 8'b0000_1001) begin
506             $display("PASS: AbortedTrans in TX_SC (status control) register is
                ↪ correct");
507         end else begin
508             $error("FAIL: Expected Tx_sc = 0x09, Received Tx_sc = 0x%h", ReadData);
509         end
510         repeat(12) @(posedge uin_hdlc.Clk); // should reach other modules too not
            ↪ sure
511         //@(posedge uin_hdlc.Clk);
512         // repeat(10) begin
513         //     @(posedge uin_hdlc.Clk);
514         //         assert (uin_hdlc.Tx == 1'b1) else begin
515         //             $error("FAIL: in idle now Expected Tx = 0b1");
516         //         end
517         // end
518       end
519 
520     end
521 
522     task Receive(int Size, int Abort, int FCSerr, int NonByteAligned, int Overflow,
        ↪ int Drop, int SkipRead);
523       logic [127:0][7:0] ReceiveData;
524       logic       [15:0] FCSBytes;
525       logic   [2:0][7:0] OverflowData;
526       logic [7:0] rx_status_reg;
527       string msg;
528       if(Abort)
529         msg = "- Abort";
530       else if(FCSerr)
531         msg = "- FCS error";
532       else if(NonByteAligned)
533         msg = "- Non-byte aligned";
534       else if(Overflow)
535         msg = "- Overflow";
536       else if(Drop)
537         msg = "- Drop";
538       else if(SkipRead)
539         msg = "- Skip read";
540       else
```

```
541         msg = "- Normal";
542       $display("****************************************************************");
543       $display("%t - Starting task Receive %s", $time, msg);
544       $display("****************************************************************");
545
546       for (int i = 0; i < Size; i++) begin
547         ReceiveData[i] = $urandom;
548       end
549       ReceiveData[Size]   = '0;
550       ReceiveData[Size+1] = '0;
551
552       //Calculate FCS bits;
553       GenerateFCSBytes(ReceiveData, Size, FCSBytes);
554       ReceiveData[Size]   = FCSBytes[7:0];
555       ReceiveData[Size+1] = FCSBytes[15:8];
556
557       //Enable FCS
558       if(!Overflow && !NonByteAligned)
559         WriteAddress(8'h2, 8'h20);
560       else
561         WriteAddress(8'h2, 8'h00);
562
563       //Generate stimulus
564       InsertFlagOrAbort(1);
565
566       MakeRxStimulus(ReceiveData, Size + 2);
567
568       if(Overflow) begin
569         OverflowData[0] = 8'h44;
570         OverflowData[1] = 8'hBB;
571         OverflowData[2] = 8'hCC;
572         MakeRxStimulus(OverflowData, 3);
573       end
574
575       //cause the frame error by inserting extra bit to frame
576       if (NonByteAligned) begin
577         @(posedge uin_hdlc.Clk);
578           uin_hdlc.Rx = 1'b1;
579       end
580
581       if(Abort) begin
582         InsertFlagOrAbort(0);
583       end else begin
584         InsertFlagOrAbort(1);
585       end
586
587       @(posedge uin_hdlc.Clk);
588       uin_hdlc.Rx = 1'b1;
589
590       repeat(8)
591         @(posedge uin_hdlc.Clk);
592
593       if(Abort) begin
594         VerifyAbortReceive(ReceiveData, Size);
595
596       end
597       else if(Overflow) begin
598         VerifyOverflowReceive(ReceiveData, Size);
```

```
599
600       end
601       else if (NonByteAligned)
602         VerifyNonByteAligned(ReceiveData, Size);
603       else if (Drop) begin
604         $display("DROP from receive");
605         VerifyDrop(ReceiveData, Size);
606       end
607       else if(!SkipRead) begin
608         VerifyNormalReceive(ReceiveData, Size);
609
610       end
611       #5000ns;
612     endtask
613
614     function void GenerateFCSBytes(logic [127:0][7:0] data, int size, output
      ↪  logic[15:0] FCSBytes);
615       logic [23:0] CheckReg;
616       CheckReg[15:8]  = data[1];
617       CheckReg[7:0]   = data[0];
618       for(int i = 2; i < size+2; i++) begin
619         CheckReg[23:16] = data[i];
620         for(int j = 0; j < 8; j++) begin
621           if(CheckReg[0]) begin
622             CheckReg[0]    = CheckReg[0] ^ 1;
623             CheckReg[1]    = CheckReg[1] ^ 1;
624             CheckReg[13:2] = CheckReg[13:2];
625             CheckReg[14]   = CheckReg[14] ^ 1;
626             CheckReg[15]   = CheckReg[15];
627             CheckReg[16]   = CheckReg[16] ^1;
628           end
629           CheckReg = CheckReg >> 1;
630         end
631       end
632       FCSBytes = CheckReg;
633       $display("%t: GenerateFCSBytes(0x%0x, %0d, 0x%04x)", $time, data, size,
       ↪  FCSBytes);
634     endfunction
635
636     covergroup cg @ (posedge uin_hdlc.Clk);
637           //OUTER SIGNALS:
638           //Address:
639                 Address : coverpoint uin_hdlc.Address {
640               bins tx_sc = {0};
641               bins tx_buff = {1};
642               bins rx_sc = {2};
643               bins rx_buff = {3};
644               bins rx_len = {4};
645               ignore_bins invalid = {[5:7]};
646                 }
647           WriteEnable : coverpoint uin_hdlc.WriteEnable {
648             bins wne = {0};
649             bins we = {1};
650           }
651           ReadEnable : coverpoint uin_hdlc.ReadEnable {
652             bins rne = {0};
653             bins re = {1};
654           }
```

```
655         DataIn : coverpoint uin_hdlc.DataIn {
656           bins datain[] = {[0:127]};
657         }
658         DataOut : coverpoint uin_hdlc.DataOut {
659           bins dataout[] = {[0:127]};
660         }
661         // TX
662         Tx : coverpoint uin_hdlc.Tx {
663           bins txz = {0};
664           bins tx1 = {1};
665         }
666         TxEN : coverpoint uin_hdlc.TxEN {
667           bins txnen = {0};
668           bins txen = {1};
669         }
670         Tx_Done : coverpoint uin_hdlc.Tx_Done {
671           bins ndone = {0};
672           bins done = {1};
673         }
674         // RX
675         Rx  : coverpoint uin_hdlc.Rx {
676           bins rxz = {0};
677           bins rx1 = {1};
678         }
679         RxEN : coverpoint uin_hdlc.RxEN {
680           bins rxnen = {0};
681           bins rxen = {1};
682         }
683         Rx_Ready : coverpoint uin_hdlc.Rx_Ready {
684           bins nready = {0};
685           bins ready = {1};
686         }
687
688         //INNER SIGNALS:
689         //RX
690         Rx_ValidFrame : coverpoint uin_hdlc.Rx_ValidFrame {
691           bins valid = {1};
692           bins invalid = {0};
693         }
694         Rx_Data : coverpoint uin_hdlc.Rx_Data {
695           bins data[] = {[7:0]};
696         }
697         Rx_AbortSignal : coverpoint uin_hdlc.Rx_AbortSignal {
698           bins abort = {1};
699           bins no_abort = {0};
700         }
701         Rx_WrBuff : coverpoint uin_hdlc.Rx_WrBuff {
702           bins write = {1};
703           bins no_write = {0};
704         }
705         Rx_EoF : coverpoint uin_hdlc.Rx_EoF {
706           bins eof = {1};
707           bins not_eof = {0};
708         }
709         Rx_FrameSize : coverpoint uin_hdlc.Rx_FrameSize {
710           bins size[] = {[7:0]};
711         }
712         Rx_Overflow : coverpoint uin_hdlc.Rx_Overflow {
```

```
713           bins overflow = {1};
714           bins no_overflow = {0};
715         }
716         Rx_FCSerr : coverpoint uin_hdlc.Rx_FCSerr {
717           bins err = {1};
718           bins no_err = {0};
719         }
720         Rx_FCSen : coverpoint uin_hdlc.Rx_FCSen {
721           bins enabled = {1};
722           bins disabled = {0};
723         }
724         Rx_DataBuffOut : coverpoint uin_hdlc.Rx_DataBuffOut {
725           bins data[] = {[7:0]};
726         }
727         Rx_RdBuff : coverpoint uin_hdlc.Rx_RdBuff {
728           bins read = {1};
729           bins no_read = {0};
730         }
731         Rx_NewByte : coverpoint uin_hdlc.Rx_NewByte {
732           bins newbyte = {1};
733           bins none = {0};
734         }
735         Rx_StartZeroDetect : coverpoint uin_hdlc.Rx_StartZeroDetect {
736           bins start = {1};
737           bins no_start = {0};
738         }
739         Rx_FlagDetect : coverpoint uin_hdlc.Rx_FlagDetect {
740           bins flag = {1};
741           bins no_flag = {0};
742         }
743         Rx_AbortDetect : coverpoint uin_hdlc.Rx_AbortDetect {
744           bins abort = {1};
745           bins no_abort = {0};
746         }
747         Rx_FrameError : coverpoint uin_hdlc.Rx_FrameError {
748           bins error = {1};
749           bins no_error = {0};
750         }
751         Rx_Drop : coverpoint uin_hdlc.Rx_Drop {
752           bins drop = {1};
753           bins keep = {0};
754         }
755         Rx_StartFCS : coverpoint uin_hdlc.Rx_StartFCS {
756           bins start = {1};
757           bins no_start = {0};
758         }
759         Rx_StopFCS : coverpoint uin_hdlc.Rx_StopFCS {
760           bins stop = {1};
761           bins no_stop = {0};
762         }
763         RxD : coverpoint uin_hdlc.RxD {
764           bins one = {1};
765           bins zero = {0};
766         }
767         ZeroDetect : coverpoint uin_hdlc.ZeroDetect {
768           bins detect = {1};
769           bins none = {0};
770         }
```

```
771         // TX
772         Tx_AbortFrame : coverpoint uin_hdlc.Tx_AbortFrame {
773           bins abort = {1};
774           bins no_abort = {0};
775         }
776         Tx_AbortedTrans : coverpoint uin_hdlc.Tx_AbortedTrans {
777           bins aborted = {1};
778           bins not_aborted = {0};
779         }
780         Tx_DataAvail : coverpoint uin_hdlc.Tx_DataAvail {
781           bins avail = {1};
782           bins not_avail = {0};
783         }
784         Tx_ValidFrame : coverpoint uin_hdlc.Tx_ValidFrame {
785           bins invalid = {0};
786           bins valid   = {1};
787         }
788 
789         Tx_WriteFCS : coverpoint uin_hdlc.Tx_WriteFCS {
790           bins no_write = {0};
791           bins write    = {1};
792         }
793 
794         Tx_InitZero : coverpoint uin_hdlc.Tx_InitZero {
795           bins no_init = {0};
796           bins init    = {1};
797         }
798         Tx_StartFCS : coverpoint uin_hdlc.Tx_StartFCS {
799           bins no_start = {0};
800           bins start    = {1};
801         }
802         Tx_RdBuff : coverpoint uin_hdlc.Tx_RdBuff {
803           bins no_read = {0};
804           bins read    = {1};
805         }
806         Tx_NewByte : coverpoint uin_hdlc.Tx_NewByte {
807           bins nob = {0};
808           bins newb  = {1};
809         }
810         Tx_FCSDone : coverpoint uin_hdlc.Tx_FCSDone {
811           bins not_done = {0};
812           bins done     = {1};
813         }
814 
815         Tx_Data : coverpoint uin_hdlc.Tx_Data {
816           bins data[] = {[7:0]};
817         }
818         Tx_Full : coverpoint uin_hdlc.Tx_Full {
819           bins not_full = {0};
820           bins full     = {1};
821         }
822         Tx_FrameSize : coverpoint uin_hdlc.Tx_FrameSize {
823           bins size[] = {[7:0]};
824         }
825         Tx_DataOutBuff : coverpoint uin_hdlc.Tx_DataOutBuff {
826           bins data[] = {[7:0]};
827         }
828         Tx_WrBuff : coverpoint uin_hdlc.Tx_WrBuff {
```

```
829           bins no_write = {0};
830           bins write    = {1};
831         }
832         Tx_Enable : coverpoint uin_hdlc.Tx_Enable {
833           bins txdis = {0};
834           bins txen  = {1};
835         }
836         Tx_DataInBuff : coverpoint uin_hdlc.Tx_DataInBuff {
837           bins data[] = {[7:0]};
838         }
839   endgroup
840   cg my_cg = new();
841 endprogram
```

B assertions_hdlc.sv

```
1  ////////////////////////////////////////////////
2  // Title:   assertions_hdlc
3  // Author:
4  // Date:
5  ////////////////////////////////////////////////
6
7  /* The assertions_hdlc module is a test module containing the concurrent
8     assertions. It is used by binding the signals of assertions_hdlc to the
9     corresponding signals in the test_hdlc testbench. This is already done in
10    bind_hdlc.sv
11
12    For this exercise you will write concurrent assertions for the Rx module:
13    - Verify that Rx_FlagDetect is asserted two cycles after a flag is received
14    - Verify that Rx_AbortSignal is asserted after receiving an abort flag
15 */
16
17 module assertions_hdlc (
18   output int   ErrCntAssertions,
19   input  logic Clk,
20   input  logic Rst,
21   input  logic Rx,
22   input  logic Rx_FlagDetect,
23   input  logic Rx_ValidFrame,
24   input  logic Rx_AbortDetect,
25   input  logic Rx_AbortSignal,
26   input  logic Rx_Overflow,
27   input  logic Rx_WrBuff,
28   input  logic Rx_EoF,
29   input  logic Rx_Ready,
30   input  logic Rx_FCSerr,
31   input  logic Rx_FCSen,
32   input  logic Rx_FrameError,
33   input  logic Tx_DataAvail,
34   input  logic Tx_Done,
35   input  logic Rx_NewByte,
36   input  logic [7:0] Rx_FrameSize
37 );
38
39   initial begin
40     ErrCntAssertions  =  0;
41   end
```

```
42
43  /*****************************************
44   *  Verify correct Rx_FlagDetect behavior  *
45   *****************************************/
46
47  sequence Rx_flag;
48    // INSERT CODE HERE
49    @(posedge Clk)
50      !Rx ##1  // 0
51      Rx [*6] ##1 // 111111
52      !Rx;      // 0
53  endsequence
54
55  // Check if flag sequence is detected
56  property RX_FlagDetect;
57    @(posedge Clk) disable iff (!Rst)
58    (Rx_flag) |-> ##2 Rx_FlagDetect;
59  endproperty
60
61  RX_FlagDetect_Assert : assert property (RX_FlagDetect) begin
62    $display("PASS: Flag detect");
63  end else begin
64    $error("Flag sequence did not generate FlagDetect");
65    ErrCntAssertions++;
66  end
67
68  /*****************************************
69   *  Verify correct Rx_AbortSignal behavior  *
70   *****************************************/
71
72  //10. Abort pattern detected during valid frame should generate Rx
    ↪  AbortSignal.
73  property RX_AbortSignalWhenValid;
74    @(posedge Clk) disable iff (!Rst)
75    ##1 (Rx_AbortDetect && Rx_ValidFrame) |=> Rx_AbortSignal;
76  endproperty
77
78  RX_AbortSignalWhenValid_Assert : assert property (RX_AbortSignalWhenValid)
    ↪  begin
79    $display("PASS. AbortSignal was generated when abort pattern was detected");
80  end else begin
81    $error("AbortSignal did not go high after AbortDetect during validframe");
82    ErrCntAssertions++;
83  end
84
85  //13. When receiving more than 128 bytes, Rx Overflow should be asserted.
86  property RX_Overflow_Detect;
87    @(posedge Clk) disable iff (!Rst || !Rx_ValidFrame)
88    $rose(Rx_ValidFrame) ##0 ($rose(Rx_NewByte) [->129]) |=> $rose(Rx_Overflow)
89  endproperty
90
91  RX_Overflow_Assert : assert property (RX_Overflow_Detect) begin
92    $display("PASS: RX_Overflow detected afrer more than 128 bytes received");
93  end else begin
94    $error("RX_Overflow did not go high after receiving more than 128 bytes");
95    ErrCntAssertions++;
96  end
97
```

```
98    // 14
99    logic [7:0] byte_counter; // Counter to track the number of bytes received
100   // Update the byte counter
101   always_ff @(posedge Clk or negedge Rst) begin
102     if (!Rst) begin
103       byte_counter <= 8'd0;
104     end else if ($fell(Rx_EoF)) begin
105       byte_counter <= 8'd0; // Reset the counter when not in a valid frame
106     end else if ($rose(Rx_NewByte)) begin
107       byte_counter <= byte_counter + 1; // Increment the counter on each new
        ↪  byte
108       $display("byte_counter %0d", byte_counter);
109     end
110   end
111
112   // 14. Rx_FrameSize should equal the exact number of bytes received in a frame
      ↪  (max. 126 bytes).
113   property RX_FrameSize_Exact;
114     @(posedge Clk) disable iff (!Rst)
115     ($rose(Rx_EoF) and !Rx_FrameError and !Rx_Overflow) |-> ##1 (Rx_FrameSize ==
        ↪  (byte_counter-2));
116   endproperty
117
118   RX_FrameSize_Exact_Assert : assert property (RX_FrameSize_Exact) begin
119     $display("PASS: Rx_FrameSize matches the exact number of bytes received.");
120   end else begin
121     $error("FAIL: Rx_FrameSize does not match the exact number of bytes received.
        ↪  Expected: %0d, Got: %0d", byte_counter-2, Rx_FrameSize);
122     ErrCntAssertions++;
123   end
124
125   //15. Rx Ready should indicate byte(s) in RX buffer is(are) ready to be read.
126   property RX_Ready;
127     @(posedge Clk) disable iff (!Rst)
128     (!Rx_Ready ##1 Rx_Ready) |-> (Rx_EoF); //Whole RX frame has been received,
        ↪  end of frame
129   endproperty
130
131   RX_Ready_Assert : assert property(RX_Ready) begin
132     $display("PASS: Bytes in RX Buffer are ready to be read");
133   end else begin
134     $error("FAIL: EoF is not asserted when buffer is ready");
135     ErrCntAssertions++;
136   end
137
138   //16. Non-byte aligned data or error in FCS checking should result in frame
      ↪  error.
139   property FCSerrFrameerr;
140     @(posedge Clk) disable iff (!Rst)
141     !Rx_FCSerr ##1 Rx_FCSerr && Rx_FCSen |=> Rx_FrameError;
142   endproperty
143
144   FCSerrFrameerr_Assert : assert property(FCSerrFrameerr) begin
145     $display("PASS. Non-byte aligned data or error in FCS checking resulted in
        ↪  frame error");
146   end else begin
147     $error("FAIL. FCSerrFrameerr_Assertion does not work");
148     ErrCntAssertions++;
```

```
  end

  //17. Tx Done should be asserted when the entire TX buffer has been read for transmission (IMMEDIATE)
  property TX_Done;
    @(posedge Clk) disable iff (!Rst)
    Tx_DataAvail ##1 !Tx_DataAvail |-> Tx_Done;
  endproperty

  TX_Done_Assert : assert property(TX_Done) begin
    $display("PASS: Imm. Tx_done asserted after no data available");
  end else begin
    $error("FAIL: Imm Tx_done not asserted properly after no data available");
    ErrCntAssertions++;
  end

endmodule
```

C Transcript Output

```
# vsim -assertdebug -c -voptargs="+acc" -coverage test_hdlc bind_hdlc -do "log -r *; run -all; coverage report -memory -cvg -details -file coverage_rep.txt; exit"
# Start time: 15:49:55 on Apr 21,2025
# ** Note: (vsim-3812) Design is being optimized...
# ** Warning: ../rtl/TxFCS.sv(13): (vopt-13314) Defaulting port 'DataBuff' kind to 'var' rather than 'wire' due to default compile option setting of -svinputport=relaxed.
# ** Note: (vopt-143) Recognized 1 FSM in module "TxFCS(fast)".
# ** Note: (vopt-143) Recognized 1 FSM in module "RxBuff(fast)".
# ** Note: (vopt-143) Recognized 1 FSM in module "TxController(fast)".
# ** Note: (vopt-143) Recognized 1 FSM in module "RxController(fast)".
# ** Note: (vopt-143) Recognized 1 FSM in module "TxBuff(fast)".
# ** Note: (vsim-12126) Error and warning message counts have been restored: Errors=0, Warnings=1.
# //  Questa Sim
# //  Version 2021.4_2 linux Dec  4 2021
# //
# //  Copyright 1991-2021 Mentor Graphics Corporation
# //  All Rights Reserved.
# //
# //  QuestaSim and its associated documentation contain trade
# //  secrets and commercial or financial information that are the property of
# //  Mentor Graphics Corporation and are privileged, confidential,
# //  and exempt from disclosure under the Freedom of Information Act,
# //  5 U.S.C. Section 552. Furthermore, this information
# //  is prohibited from disclosure under the Trade Secrets Act,
# //  18 U.S.C. Section 1905.
# //
# Loading sv_std.std
# Loading work.test_hdlc(fast)
# Loading work.in_hdlc(fast__1)
# Loading work.Hdlc(fast)
# Loading work.AddressIF(fast)
# Loading work.TxController(fast)
# Loading work.TxBuff(fast)
# Loading work.TxFCS(fast)
```

```
# Loading work.TxChannel(fast)
# Loading work.RxController(fast)
# Loading work.RxBuff(fast)
# Loading work.RxFCS(fast)
# Loading work.RxChannel(fast)
# Loading work.testPr_hdlc(fast)
# Loading work.assertions_hdlc(fast)
# Loading work.bind_hdlc(fast)
# ** Warning: (vsim-8634) Code was not compiled with coverage options.
# log -r *
#  run -all
# ****************************************************************
#                    0 - Starting Test Program
# ****************************************************************
# ****************************************************************
#                 1000 - Starting task Receive - Normal
# ****************************************************************
#                 1000: GenerateFCSBytes(0xx00003c135d73b7c48caa2886, 10, 0xd0bc)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received normal sequence
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# ****************************************************************
#                78750 - Starting task Receive - Abort
# ****************************************************************
#                78750:
↪  GenerateFCSBytes(0xx000050827d55ac4ed5c97bc02309e409eb7b8c912aecfaf68960da798a855d639407e1e9f2
↪  40, 0x865a)
# PASS: Flag detect
# PASS. AbortSignal was generated when abort pattern was detected
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received abort sequence
# PASS. Rx_Buff has correct data when abort
# ****************************************************************
#               269250 - Starting task Receive - Overflow
# ****************************************************************
#               269250:
↪  GenerateFCSBytes(0x88493a916c411fb46a15af13aad8fe5caf301fa8c74d8baa85500717d8d65d9a6083c3866d3
↪  126, 0x4326)
# PASS: Flag detect
# PASS: RX_Overflow detected afrer more than 128 bytes received
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: RX Status/Control received overflow sequence
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# ************************************************************
#                944750 - Starting task Receive - Normal
# ************************************************************
#                944750:
↪  GenerateFCSBytes(0x432688493a916c411fb46a15af13aad8fe5caf301fa8c74d8baa85500717d8d65d9a6083c38
↪  45, 0xf213)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received normal sequence
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
```

```
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# ************************************************************
#              1200750 - Starting task Receive - Normal
# ************************************************************
#              1200750:
↪  GenerateFCSBytes(0xf4fcb580355ec5c8b5043d7558e116f854aa458dfd6da17125eb78524fbea9b51e5a73924e9
↪  126, 0x63b6)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received normal sequence
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
```

```
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
```

```
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# *************************************************************
#              1867250 - Starting task Receive - Abort
# *************************************************************
#              1867250:
↪ GenerateFCSBytes(0x63b6f4fc0000e6a3c78ff18b55fb22598a22ea4ef7e6b76efb93f996c20e8910b3e837cf09a
↪ 122, 0x33d0)
# PASS: Flag detect
# PASS. AbortSignal was generated when abort pattern was detected
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received abort sequence
# PASS. Rx_Buff has correct data when abort
# *************************************************************
#              2394750 - Starting task Receive - Overflow
# *************************************************************
#              2394750:
↪ GenerateFCSBytes(0x1606e4cd8393aba85966dd906751ffe89900adbf9ee7aa45840b52f16c8575d2dd244e46d97
↪ 126, 0xb8d8)
# PASS: Flag detect
# PASS: RX_Overflow detected afrer more than 128 bytes received
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: RX Status/Control received overflow sequence
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
```

```
# PASS. Rx_Buff has correct data when overflow
# PASS. Rx_Buff has correct data when overflow
# *************************************************************
#              3072750 - Starting task Receive - Normal
# *************************************************************
#              3072750:
↪  GenerateFCSBytes(0xb8d81606e4cd8393aba85966dd906751ffe89900adbf9ee7aa45840b52f16c8575d2dd244e4
↪  25, 0xda31)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received normal sequence
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# *************************************************************
#              3227250 - Starting task Receive - Normal
# *************************************************************
#              3227250:
↪  GenerateFCSBytes(0xb8d81606e4cd8393aba85966dd906751ffe89900adbf9ee7aa45840b52f16c8575d2dd244e4
↪  47, 0x9771)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# PASS: RX Status/Control received normal sequence
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
```

```
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# PASS. Rx_Buff has correct data when normal
# *************************************************************
#              3492750 - Starting task Receive - Drop
# *************************************************************
#              3492750:
↪ GenerateFCSBytes(0xb8d81606e4cd8393aba85966dd906751ffe89900adbf9ee7aa45840b52f16c8575d2dd244e4
↪ 25, 0x055b)
# PASS: Flag detect
# PASS: Flag detect
# PASS: Bytes in RX Buffer are ready to be read
# PASS: Rx_FrameSize matches the exact number of bytes received.
# DROP from receive
# PASS. Rx_Buff has correct data when drop
# *************************************************************
#              3623750 - Starting task Receive - Non-byte aligned
# *************************************************************
#              3623750:
↪ GenerateFCSBytes(0xb8d81606e4cd8393aba85966dd906751ffe89900adbf9ee7aa45840b52f16c8575d2dd244e4
↪ 47, 0xfcb9)
# PASS: Flag detect
# PASS: Flag detect
```

```
# PASS: Rx status control register contains correct Frame Error Bit
# PASS: Rx_Buff has correct data when non byte aligned
#               3843250: Transmit(0)
#               3843250: Cannot send more bytes that what Tx Buffer can hold!
#               5843250: Transmit(3)
#               5844250: TX transmission: Filling Tx Buffer
#               5851000: GenerateFCSBytes(0x1c3cf9, 3, 0xf8c0)
#               5851000: Register Monitor: Detected start of TX transmission.
    ↪  Expecting FCS: 0xf8c0
#               5851250: TX transmission: Started
#               5875250: TX transmission: TX Buffer Done
#               5875250: TX transmission: Done (flag detected)
# PASS: Imm. Tx_done asserted after no data available
#               5879250: TX Monitor: Going to state RECEIVING
# PASS: Correct TX output according to written TX buffer.
#               5883750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#               5900250: TX Monitor: Removing a zero
# PASS. Tx_done asserted after tranmission is done
#               5904250: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
#               6880250: Transmit(4)
#               6881250: TX transmission: Filling Tx Buffer
#               6890000: GenerateFCSBytes(0x2d0304e3, 4, 0xa8b6)
#               6890000: Register Monitor: Detected start of TX transmission.
    ↪  Expecting FCS: 0xa8b6
#               6890250: TX transmission: Started
#               6922250: TX Monitor: Going to state RECEIVING
#               6923250: TX transmission: TX Buffer Done
# PASS: Imm. Tx_done asserted after no data available
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# 6944250: TX transmission: Sending abort after 16 number of cycles
# read data from the tx_statusControl is 09
# PASS: AbortedTrans in TX_SC (status control) register is correct
# PASS: Abort pattern 0xfe received after abort set in TX_SC reg
# PASS: Tx AbortedTrans is set correctly after abort
#               6950750: TX Monitor: Going to state WAITING_FOR_FLAG
#               6950750: TX transmission: Done (flag detected)
#               6955750: Transmit(8)
#               6956750: TX transmission: Filling Tx Buffer
#               6973500: GenerateFCSBytes(0xd8201c74adb6aae9, 8, 0x0ab0)
#               6973500: Register Monitor: Detected start of TX transmission.
    ↪  Expecting FCS: 0x0ab0
#               6973750: TX transmission: Started
#               7021750: TX Monitor: Going to state RECEIVING
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#               7038750: TX transmission: TX Buffer Done
# PASS: Imm. Tx_done asserted after no data available
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# 7052750: TX transmission: Sending abort after 16 number of cycles
```

```
706 # PASS: Correct TX output according to written TX buffer.
707 # read data from the tx_statusControl is 09
708 # PASS: AbortedTrans in TX_SC (status control) register is correct
709 # PASS: Abort pattern 0xfe received after abort set in TX_SC reg
710 # PASS: Tx AbortedTrans is set correctly after abort
711 #              7059250: TX Monitor: Going to state WAITING_FOR_FLAG
712 #              7059250: TX transmission: Done (flag detected)
713 #              7064250: Transmit(16)
714 #              7065250: TX transmission: Filling Tx Buffer
715 #              7098000: GenerateFCSBytes(0xe57823bfaa816f65856c095f37a40729, 16,
    ↪ 0x8d78)
716 #              7098000: Register Monitor: Detected start of TX transmission.
    ↪ Expecting FCS: 0x8d78
717 #              7098250: TX transmission: Started
718 #              7178250: TX Monitor: Going to state RECEIVING
719 # PASS: Correct TX output according to written TX buffer.
720 # PASS: Correct TX output according to written TX buffer.
721 # PASS: Correct TX output according to written TX buffer.
722 # PASS: Correct TX output according to written TX buffer.
723 #              7197250: TX Monitor: Removing a zero
724 # PASS: Correct TX output according to written TX buffer.
725 # PASS: Correct TX output according to written TX buffer.
726 # PASS: Correct TX output according to written TX buffer.
727 # PASS: Correct TX output according to written TX buffer.
728 # PASS: Correct TX output according to written TX buffer.
729 # PASS: Correct TX output according to written TX buffer.
730 # PASS: Correct TX output according to written TX buffer.
731 # PASS: Correct TX output according to written TX buffer.
732 #              7228250: TX transmission: TX Buffer Done
733 # PASS: Imm. Tx_done asserted after no data available
734 #              7229250: TX Monitor: Removing a zero
735 # PASS: Correct TX output according to written TX buffer.
736 # PASS: Correct TX output according to written TX buffer.
737 # PASS: Correct TX output according to written TX buffer.
738 # PASS: Correct TX output according to written TX buffer.
739 # PASS. Tx_done asserted after tranmission is done
740 #              7255250: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
741 #              7255250: TX transmission: Done (flag detected)
742 #              7260250: Transmit(32)
743 #              7261250: TX transmission: Filling Tx Buffer
744 #              7326000:
    ↪ GenerateFCSBytes(0x2503cfa0c6ea2069830f880942abdc2287e17f8311b16f3644e51cc2541defc4,
    ↪ 32, 0xbb27)
745 #              7326000: Register Monitor: Detected start of TX transmission.
    ↪ Expecting FCS: 0xbb27
746 #              7326250: TX transmission: Started
747 #              7470250: TX Monitor: Going to state RECEIVING
748 # PASS: Correct TX output according to written TX buffer.
749 #              7476250: TX Monitor: Removing a zero
750 # PASS: Correct TX output according to written TX buffer.
751 # 7480750: TX transmission: Sending abort after 16 number of cycles
752 #              7482750: TX transmission: TX Buffer Done
753 # PASS: Imm. Tx_done asserted after no data available
754 # read data from the tx_statusControl is 09
755 # PASS: AbortedTrans in TX_SC (status control) register is correct
756 # PASS: Abort pattern 0xfe received after abort set in TX_SC reg
757 # PASS: Tx AbortedTrans is set correctly after abort
758 #              7487250: TX Monitor: Going to state WAITING_FOR_FLAG
```

```
#              7487250: TX transmission: Done (flag detected)
#              7492250: Transmit(32)
#              7493250: TX transmission: Filling Tx Buffer
#              7558000:
↪ GenerateFCSBytes(0x88fa08a9f988989663f4bc66fe8f55188d0cb3dee5ff344dfdd14d024a4243fc,
↪ 32, 0x9d74)
#              7558000: Register Monitor: Detected start of TX transmission.
↪ Expecting FCS: 0x9d74
#              7558250: TX transmission: Started
#              7702250: TX Monitor: Going to state RECEIVING
#              7706250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7734750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7746250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7783250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7797250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7817250: TX Monitor: Removing a zero
#              7818250: TX transmission: TX Buffer Done
# PASS: Imm. Tx_done asserted after no data available
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#              7829750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS. Tx_done asserted after tranmission is done
#              7845750: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
#              7845750: TX transmission: Done (flag detected)
#              7850750: Transmit(64)
#              7851750: TX transmission: Filling Tx Buffer
```

```
812 #             7980500:
    ↪ GenerateFCSBytes(0x76d22f1c260be25ed1ef7218b892df414bbc4f7b5ef59cde8be59858099727a64dac84d255c
    ↪ 64, 0xee8e)
813 #             7980500: Register Monitor: Detected start of TX transmission.
    ↪ Expecting FCS: 0xee8e
814 #             7980750: TX transmission: Started
815 #             8252750: TX Monitor: Going to state RECEIVING
816 # PASS: Correct TX output according to written TX buffer.
817 # PASS: Correct TX output according to written TX buffer.
818 # PASS: Correct TX output according to written TX buffer.
819 # PASS: Correct TX output according to written TX buffer.
820 # PASS: Correct TX output according to written TX buffer.
821 # PASS: Correct TX output according to written TX buffer.
822 #             8279750: TX Monitor: Removing a zero
823 # PASS: Correct TX output according to written TX buffer.
824 # PASS: Correct TX output according to written TX buffer.
825 # PASS: Correct TX output according to written TX buffer.
826 # PASS: Correct TX output according to written TX buffer.
827 # PASS: Correct TX output according to written TX buffer.
828 #             8298250: TX Monitor: Removing a zero
829 # PASS: Correct TX output according to written TX buffer.
830 #             8303750: TX Monitor: Removing a zero
831 # PASS: Correct TX output according to written TX buffer.
832 # PASS: Correct TX output according to written TX buffer.
833 #             8311750: TX Monitor: Removing a zero
834 # PASS: Correct TX output according to written TX buffer.
835 # PASS: Correct TX output according to written TX buffer.
836 # PASS: Correct TX output according to written TX buffer.
837 # PASS: Correct TX output according to written TX buffer.
838 # PASS: Correct TX output according to written TX buffer.
839 #             8332250: TX Monitor: Removing a zero
840 # PASS: Correct TX output according to written TX buffer.
841 # PASS: Correct TX output according to written TX buffer.
842 # PASS: Correct TX output according to written TX buffer.
843 #             8344250: TX Monitor: Removing a zero
844 # PASS: Correct TX output according to written TX buffer.
845 #             8350750: TX Monitor: Removing a zero
846 # PASS: Correct TX output according to written TX buffer.
847 # PASS: Correct TX output according to written TX buffer.
848 # PASS: Correct TX output according to written TX buffer.
849 # PASS: Correct TX output according to written TX buffer.
850 # PASS: Correct TX output according to written TX buffer.
851 # PASS: Correct TX output according to written TX buffer.
852 # PASS: Correct TX output according to written TX buffer.
853 # PASS: Correct TX output according to written TX buffer.
854 # PASS: Correct TX output according to written TX buffer.
855 # PASS: Correct TX output according to written TX buffer.
856 # PASS: Correct TX output according to written TX buffer.
857 # PASS: Correct TX output according to written TX buffer.
858 # PASS: Correct TX output according to written TX buffer.
859 # PASS: Correct TX output according to written TX buffer.
860 # PASS: Correct TX output according to written TX buffer.
861 # PASS: Correct TX output according to written TX buffer.
862 #             8413750: TX Monitor: Removing a zero
863 # PASS: Correct TX output according to written TX buffer.
864 # PASS: Correct TX output according to written TX buffer.
865 # PASS: Correct TX output according to written TX buffer.
866 # PASS: Correct TX output according to written TX buffer.
```

```
867 # PASS: Correct TX output according to written TX buffer.
868 # PASS: Correct TX output according to written TX buffer.
869 # PASS: Correct TX output according to written TX buffer.
870 # PASS: Correct TX output according to written TX buffer.
871 # PASS: Correct TX output according to written TX buffer.
872 # PASS: Correct TX output according to written TX buffer.
873 #              8455750: TX Monitor: Removing a zero
874 # PASS: Correct TX output according to written TX buffer.
875 # PASS: Correct TX output according to written TX buffer.
876 # PASS: Correct TX output according to written TX buffer.
877 # PASS: Correct TX output according to written TX buffer.
878 # PASS: Correct TX output according to written TX buffer.
879 # PASS: Correct TX output according to written TX buffer.
880 # PASS: Correct TX output according to written TX buffer.
881 # PASS: Correct TX output according to written TX buffer.
882 # PASS: Correct TX output according to written TX buffer.
883 #              8490750: TX Monitor: Removing a zero
884 # PASS: Correct TX output according to written TX buffer.
885 # PASS: Correct TX output according to written TX buffer.
886 #              8498750: TX transmission: TX Buffer Done
887 # PASS: Imm. Tx_done asserted after no data available
888 # PASS: Correct TX output according to written TX buffer.
889 # PASS: Correct TX output according to written TX buffer.
890 # PASS: Correct TX output according to written TX buffer.
891 # PASS: Correct TX output according to written TX buffer.
892 # PASS. Tx_done asserted after tranmission is done
893 #              8525750: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
894 #              8525750: TX transmission: Done (flag detected)
895 #              8530750: Transmit(64)
896 #              8531750: TX transmission: Filling Tx Buffer
897 #              8660500:
    ↪ GenerateFCSBytes(0x592544fcdf8e036ed837107818e34b53d0bd1e7711fd873de463864b38209060280f46c4485
    ↪ 64, 0x83f2)
898 #              8660500: Register Monitor: Detected start of TX transmission.
    ↪ Expecting FCS: 0x83f2
899 #              8660750: TX transmission: Started
900 #              8932750: TX Monitor: Going to state RECEIVING
901 # PASS: Correct TX output according to written TX buffer.
902 #              8940250: TX Monitor: Removing a zero
903 # PASS: Correct TX output according to written TX buffer.
904 # PASS: Correct TX output according to written TX buffer.
905 # PASS: Correct TX output according to written TX buffer.
906 # PASS: Correct TX output according to written TX buffer.
907 # PASS: Correct TX output according to written TX buffer.
908 # PASS: Correct TX output according to written TX buffer.
909 #              8962250: TX Monitor: Removing a zero
910 # PASS: Correct TX output according to written TX buffer.
911 # PASS: Correct TX output according to written TX buffer.
912 # PASS: Correct TX output according to written TX buffer.
913 # PASS: Correct TX output according to written TX buffer.
914 # PASS: Correct TX output according to written TX buffer.
915 # PASS: Correct TX output according to written TX buffer.
916 # PASS: Correct TX output according to written TX buffer.
917 # PASS: Correct TX output according to written TX buffer.
918 # PASS: Correct TX output according to written TX buffer.
919 # PASS: Correct TX output according to written TX buffer.
920 # PASS: Correct TX output according to written TX buffer.
921 #              9007750: TX Monitor: Removing a zero
```

```
922 # PASS: Correct TX output according to written TX buffer.
923 # PASS: Correct TX output according to written TX buffer.
924 # PASS: Correct TX output according to written TX buffer.
925 # PASS: Correct TX output according to written TX buffer.
926 # PASS: Correct TX output according to written TX buffer.
927 # PASS: Correct TX output according to written TX buffer.
928 #              9034250: TX Monitor: Removing a zero
929 # PASS: Correct TX output according to written TX buffer.
930 # PASS: Correct TX output according to written TX buffer.
931 # PASS: Correct TX output according to written TX buffer.
932 # PASS: Correct TX output according to written TX buffer.
933 # PASS: Correct TX output according to written TX buffer.
934 # PASS: Correct TX output according to written TX buffer.
935 # PASS: Correct TX output according to written TX buffer.
936 # PASS: Correct TX output according to written TX buffer.
937 # PASS: Correct TX output according to written TX buffer.
938 # PASS: Correct TX output according to written TX buffer.
939 # PASS: Correct TX output according to written TX buffer.
940 # PASS: Correct TX output according to written TX buffer.
941 # PASS: Correct TX output according to written TX buffer.
942 # PASS: Correct TX output according to written TX buffer.
943 # PASS: Correct TX output according to written TX buffer.
944 # PASS: Correct TX output according to written TX buffer.
945 # PASS: Correct TX output according to written TX buffer.
946 # PASS: Correct TX output according to written TX buffer.
947 #              9106750: TX Monitor: Removing a zero
948 # PASS: Correct TX output according to written TX buffer.
949 # PASS: Correct TX output according to written TX buffer.
950 # PASS: Correct TX output according to written TX buffer.
951 # PASS: Correct TX output according to written TX buffer.
952 # PASS: Correct TX output according to written TX buffer.
953 # PASS: Correct TX output according to written TX buffer.
954 # PASS: Correct TX output according to written TX buffer.
955 # PASS: Correct TX output according to written TX buffer.
956 # PASS: Correct TX output according to written TX buffer.
957 # PASS: Correct TX output according to written TX buffer.
958 # PASS: Correct TX output according to written TX buffer.
959 # PASS: Correct TX output according to written TX buffer.
960 # PASS: Correct TX output according to written TX buffer.
961 # PASS: Correct TX output according to written TX buffer.
962 # PASS: Correct TX output according to written TX buffer.
963 # PASS: Correct TX output according to written TX buffer.
964 # PASS: Correct TX output according to written TX buffer.
965 #              9173750: TX Monitor: Removing a zero
966 # PASS: Correct TX output according to written TX buffer.
967 #              9177250: TX transmission: TX Buffer Done
968 # PASS: Imm. Tx_done asserted after no data available
969 #              9179750: TX Monitor: Removing a zero
970 # PASS: Correct TX output according to written TX buffer.
971 # PASS: Correct TX output according to written TX buffer.
972 # PASS: Correct TX output according to written TX buffer.
973 # PASS: Correct TX output according to written TX buffer.
974 #              9197250: TX Monitor: Removing a zero
975 # PASS. Tx_done asserted after tranmission is done
976 #              9204750: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
977 #              9204750: TX transmission: Done (flag detected)
978 #              9209750: Transmit(126)
979 #              9210750: TX transmission: Filling Tx Buffer
```

```
980 # PASS. TX Full asserted after receiving more or 126 bytes
981 #             9464500:
    ↪ GenerateFCSBytes(0xe11690e7f2300f9b1c3f0cdde0f6c6301246879b47e166d631591b84680534e0b6006441b33
    ↪ 126, 0x1ca1)
982 #             9464500: Register Monitor: Detected start of TX transmission.
    ↪ Expecting FCS: 0x1ca1
983 #             9464750: TX transmission: Started
984 #             9984750: TX Monitor: Going to state RECEIVING
985 # PASS: Correct TX output according to written TX buffer.
986 # PASS: Correct TX output according to written TX buffer.
987 # PASS: Correct TX output according to written TX buffer.
988 # PASS: Correct TX output according to written TX buffer.
989 # PASS: Correct TX output according to written TX buffer.
990 # PASS: Correct TX output according to written TX buffer.
991 # PASS: Correct TX output according to written TX buffer.
992 # PASS: Correct TX output according to written TX buffer.
993 # PASS: Correct TX output according to written TX buffer.
994 # PASS: Correct TX output according to written TX buffer.
995 # PASS: Correct TX output according to written TX buffer.
996 # PASS: Correct TX output according to written TX buffer.
997 # PASS: Correct TX output according to written TX buffer.
998 # PASS: Correct TX output according to written TX buffer.
999 # PASS: Correct TX output according to written TX buffer.
1000 # PASS: Correct TX output according to written TX buffer.
1001 # PASS: Correct TX output according to written TX buffer.
1002 # PASS: Correct TX output according to written TX buffer.
1003 # PASS: Correct TX output according to written TX buffer.
1004 # PASS: Correct TX output according to written TX buffer.
1005 # PASS: Correct TX output according to written TX buffer.
1006 # PASS: Correct TX output according to written TX buffer.
1007 # PASS: Correct TX output according to written TX buffer.
1008 # PASS: Correct TX output according to written TX buffer.
1009 # PASS: Correct TX output according to written TX buffer.
1010 #            10085750: TX Monitor: Removing a zero
1011 # PASS: Correct TX output according to written TX buffer.
1012 # PASS: Correct TX output according to written TX buffer.
1013 # PASS: Correct TX output according to written TX buffer.
1014 #            10099250: TX Monitor: Removing a zero
1015 # PASS: Correct TX output according to written TX buffer.
1016 #            10104250: TX Monitor: Removing a zero
1017 # PASS: Correct TX output according to written TX buffer.
1018 # PASS: Correct TX output according to written TX buffer.
1019 # PASS: Correct TX output according to written TX buffer.
1020 # PASS: Correct TX output according to written TX buffer.
1021 # PASS: Correct TX output according to written TX buffer.
1022 # PASS: Correct TX output according to written TX buffer.
1023 # PASS: Correct TX output according to written TX buffer.
1024 # PASS: Correct TX output according to written TX buffer.
1025 # PASS: Correct TX output according to written TX buffer.
1026 # PASS: Correct TX output according to written TX buffer.
1027 # PASS: Correct TX output according to written TX buffer.
1028 # PASS: Correct TX output according to written TX buffer.
1029 # PASS: Correct TX output according to written TX buffer.
1030 # PASS: Correct TX output according to written TX buffer.
1031 # PASS: Correct TX output according to written TX buffer.
1032 # PASS: Correct TX output according to written TX buffer.
1033 # PASS: Correct TX output according to written TX buffer.
1034 # PASS: Correct TX output according to written TX buffer.
```

```
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10235250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10243250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10255250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10260250: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10316750: TX Monitor: Removing a zero
#             10319750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
```

```
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10410750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10456750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10468750: TX Monitor: Removing a zero
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
#             10479750: TX Monitor: Removing a zero
#             10480250: TX transmission: TX Buffer Done
# PASS: Imm. Tx_done asserted after no data available
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS: Correct TX output according to written TX buffer.
# PASS. Tx_done asserted after tranmission is done
#             10507250: TX Monitor: Going from ENDFRAME to state WAITING_FOR_FLAG
#             10507250: TX transmission: Done (flag detected)
# ================================================================
#             10512250 - Finishing Test Program
# ================================================================
# ** Note: $stop    : ./testPr_hdlc.sv(332)
#    Time: 10512250 ns  Iteration: 1  Instance: /test_hdlc/u_testPr
# Break in Module testPr_hdlc at ./testPr_hdlc.sv line 332
# Stopped at ./testPr_hdlc.sv line 332
#  coverage report -memory -cvg -details -file coverage_rep.txt
# ** Warning: (vsim-19053) Option '-file' has been deprecated. Please use option
  ↪  '-output' instead.
#  exit
# ********************************
# *                              *
# *         Assertion Errors: 0          *
# *                              *
# ********************************
# End time: 15:50:02 on Apr 21,2025, Elapsed time: 0:00:07
# Errors: 0, Warnings: 3
```